



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of the UGC Act, 1956)

VELLORE CAMPUS



Aryan Nair

24BDS0320

HOSTELLER

App. No: 2024164996

Blood Group: O+

Official Address:

Vellore Institute of Technology,
Vellore - 632 014, Tamilnadu, India.

Ph : 0416 224 3091

Fax : 0416 224 3092

Address:

MRRA 35D, MOOLEPADAM ROAD,
VAZHAKALA, KAKKANADU,
ERNAKULAM, 682030, KERALA,
INDIA

Contact No.:

9847035186

Valid Upto: **JUL-2028**

T. Jayar

Issuing Authority

Holder's Signature



24BDS0320

Website : www.vit.ac.in

Task 3

MPMC LAB

Registration Number: 24BDS0320

Name: Aryan Nair

Experiment Number: 3

Date: 12 March 2026

AIM:

To write, compile and simulate an 8051 microcontroller program using Keil μ Vision software and verify the output using the simulator.

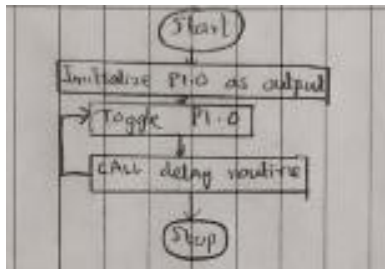
APPARTUS REQUIRED:

Keil μ Vision software

ALGORITHM:

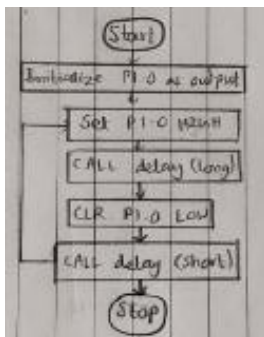
1)

a)



1. Start
2. Select pin P1.0 as output.
3. Toggle state of P1.0.
4. Call a delay routine to maintain required time interval.
5. Toggle P1.0 again.
6. Repeat steps 3–5 continuously to generate square wave.
7. Stop

b)

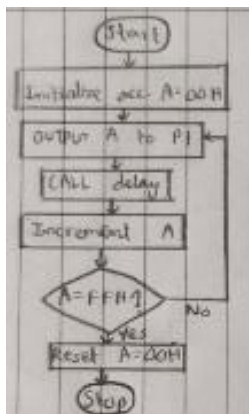


1. Start
2. Select pin P1.0 as output.
3. Set P1.0 to logic HIGH.

4. Call delay routine (long delay).
5. Clear P1.0 to logic LOW.
6. Call another delay routine (short delay).
7. Repeat steps 3–6 continuously to produce rectangular waveform.
8. Stop

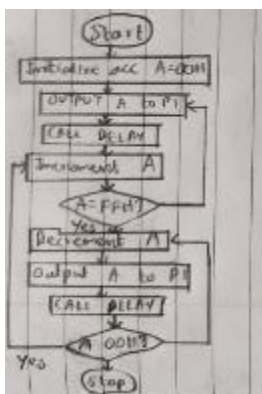
2)

a)



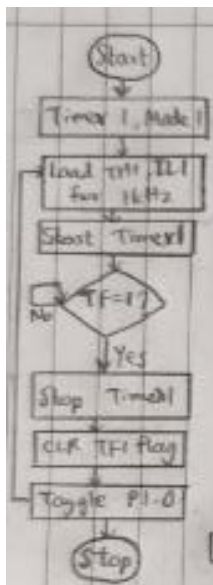
1. Start
2. Initialize accumulator A with 00H.
3. Send value of accumulator to output port P1.
4. Call delay routine to control waveform speed.
5. Increment accumulator value.
6. Repeat steps 3–5 till accumulator reaches FFH.
7. After reaching FFH, reset accumulator to 00H.
8. Repeat process continuously to produce sawtooth waveform.
9. Stop

b)



1. Start
2. Initialize accumulator A with value 00H.
3. Send value of accumulator to port P1.
4. Call delay routine.
5. Increment accumulator value.
6. Repeat steps 3–5 till accumulator reaches FFH.
7. Decrement accumulator value.
8. Send updated value to port P1.
9. Call delay routine.
10. Repeat steps 7–9 till accumulator reaches 00H.
11. Continue process to generate a triangular waveform.
12. Stop

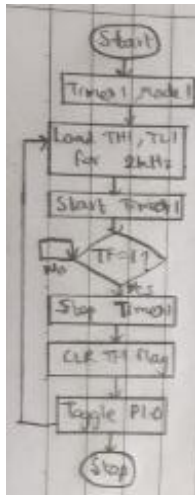
3)



1. Start
2. Configure Timer1 in mode 1.
3. Load appropriate initial values in TH1 and TL1 for 1 kHz frequency.
4. Start Timer1.
5. Wait till timer overflow flag TF1 becomes 1.
6. Stop Timer1 and clear TF1 flag.
7. Toggle output pin P1.0.
8. Reload timer values.
9. Repeat steps 4–8 continuously to generate a 1 kHz square wave.

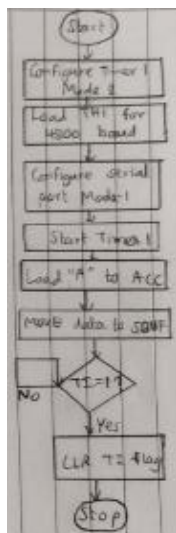
10.Stop

4)



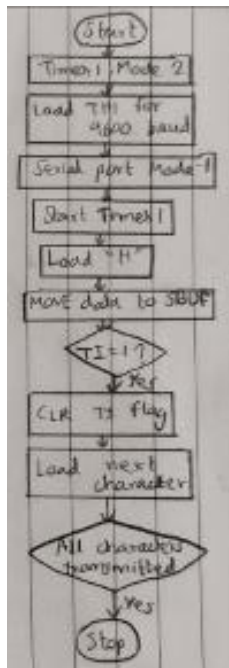
1. Start
2. Configure Timer1 in mode 1.
3. Load TH1 and TL1 with values required for 2 kHz frequency.
4. Start Timer1.
5. Wait for timer overflow ($TF1 = 1$).
6. Stop Timer1 and clear TF1 flag.
7. Toggle output pin P1.0.
8. Reload timer registers with initial values.
9. Repeat steps 4–8 continuously to generate a 2 kHz square wave.
- 10.Stop

5)



1. Start
2. Configure Timer-1 in mode-2 (auto reload mode).
3. Load TH1 with the value required to generate 4800 baud rate.
4. Configure serial port in mode-1 (8-bit UART) using SCON register.
5. Start Timer-1.
6. Load ASCII value of character 'A' into accumulator.
7. Move accumulator value into SBUF to start transmission.
8. Wait till transmission complete flag TI = 1.
9. Clear TI flag.
10. Repeat steps 6–9 continuously to transmit character repeatedly.
11. Stop

6)



1. Start
2. Configure Timer-1 in mode-2 (auto reload).
3. Load TH1 with value required for 9600 baud rate.
4. Configure serial port in mode-1 (8-bit UART, 1 stop bit).
5. Start Timer-1.
6. Load first character 'H' into accumulator.
7. Move character into SBUF for serial transmission.
8. Wait till transmission complete flag TI = 1.
9. Clear TI flag.

10. Load next character 'E' and repeat transmission.
11. Continue transmitting 'L', 'L', 'O' sequentially.
12. After transmitting last character, repeat the sequence from 'H'.
13. Continue transmitting the word HELLO repeatedly.
14. Stop

CODE:

1)

```

a) ORG 0000H
START:
CPL P1.0
ACALL DELAY
SJMP START
DELAY:
MOV R2, #200
D1: MOV R3, #255
D2: DJNZ R3, D2
RET
END

```

```

ORG 0000H
b) START:
SETB P1.0
ACALL DELAY1
CLR P1.0
ACALL DELAY2
SJMP START
DELAY1:
MOV R2, #200
L1: MOV R3, #255
L2: DJNZ R3, L2
DJNZ R2, L1
RET
DELAY2:
MOV R2, #80
L3: MOV R3, #255
L4: DJNZ R3, L4
DJNZ R2, L3
RET
END

```

2)

```

a) ORG 0000H
MOV A, #00H
NEXT:
MOV P1, A
INX A
ACALL DELAY
SJMP NEXT
DELAY:
MOV R2, #50
D1: MOV R3, #200
D2: DJNZ R3, D2
DJNZ R2, D1
RET
END

```

```

ORG 0000H
b) START:
SETB P1.0
ACALL DELAY1
CLR P1.0
ACALL DELAY2
SJMP START
DELAY1:
MOV R2, #200
L1: MOV R3, #255
L2: DJNZ R3, L2
DJNZ R2, L1
RET
DELAY2:
MOV R2, #80
L3: MOV R3, #255
L4: DJNZ R3, L4
DJNZ R2, L3
RET
END

```

3)

```

ORG 0000H
MOV TMOD, #10H
AGAIN:
MOV TH1, #0FCH
MOV TL1, #66H
SETB TR1
WAIT:
JNB TFI, WAIT
CLR TR1
CLR TFI
CPL P1.0
SJMP AGAIN
END

```


4)

```
ORG 0000H
MOV TMOD, #10H
AGAIN:
MOV TH1, #0FEH
MOV TL1, #00H
SETB TR1
WAIT:
JNB TF1, WAIT
CLR TR1
CLR TF1
CPL P1.0
SJMP AGAIN
END
```

5)

```
ORG 0000H
MOV TMOD, #20H
MOV TH1, #-6
MOV SCON, #50H
SETB TR1
AGAIN: MOV SBUF, # "A"
HERE: JNB TI, HERE
      CLR TI
      SJMP AGAIN
END
```

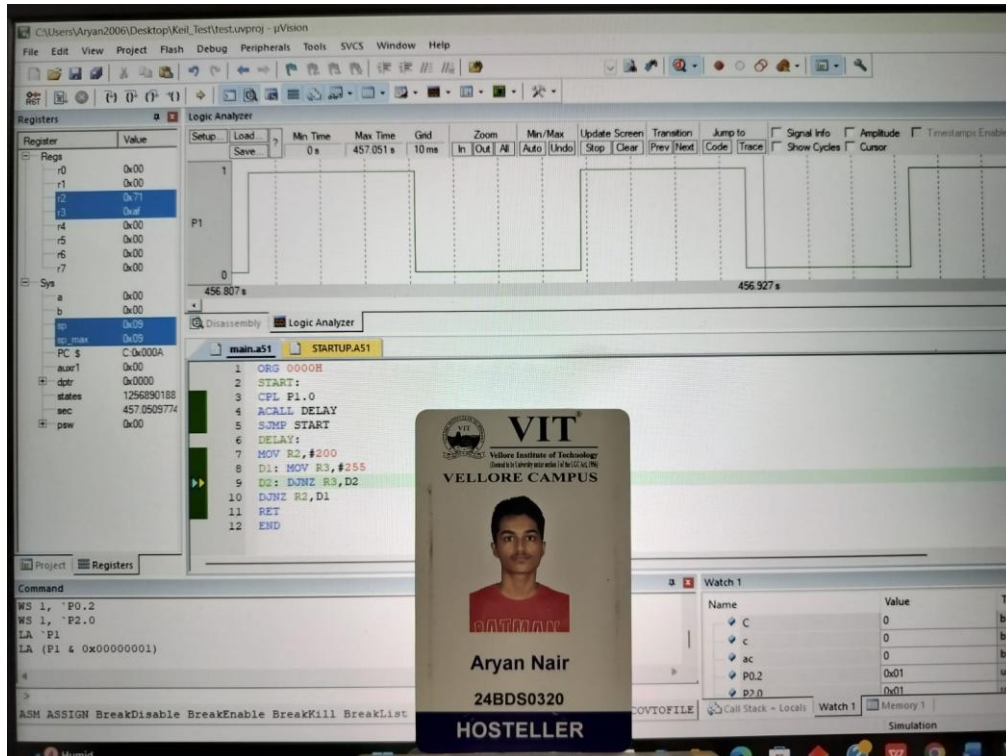
6)

```
ORG 0000H
MOV TMOD, #20H
MOV TH1, #-3
MOV SCON, #50H
SETB TR1
AGAIN: MOV A, # "H"
      ACALL TRANS
      MOV A, # "E"
      ACALL TRANS
      MOV A, # "L"
      ACALL TRANS
      MOV A, # "L"
      ACALL TRANS
      MOV A, # "O"
      ACALL TRANS
      SJMP AGAIN
TRANS: MOV SBUF, A
HERE: JNB TI, HERE
      CLR TI
      RET
END
```

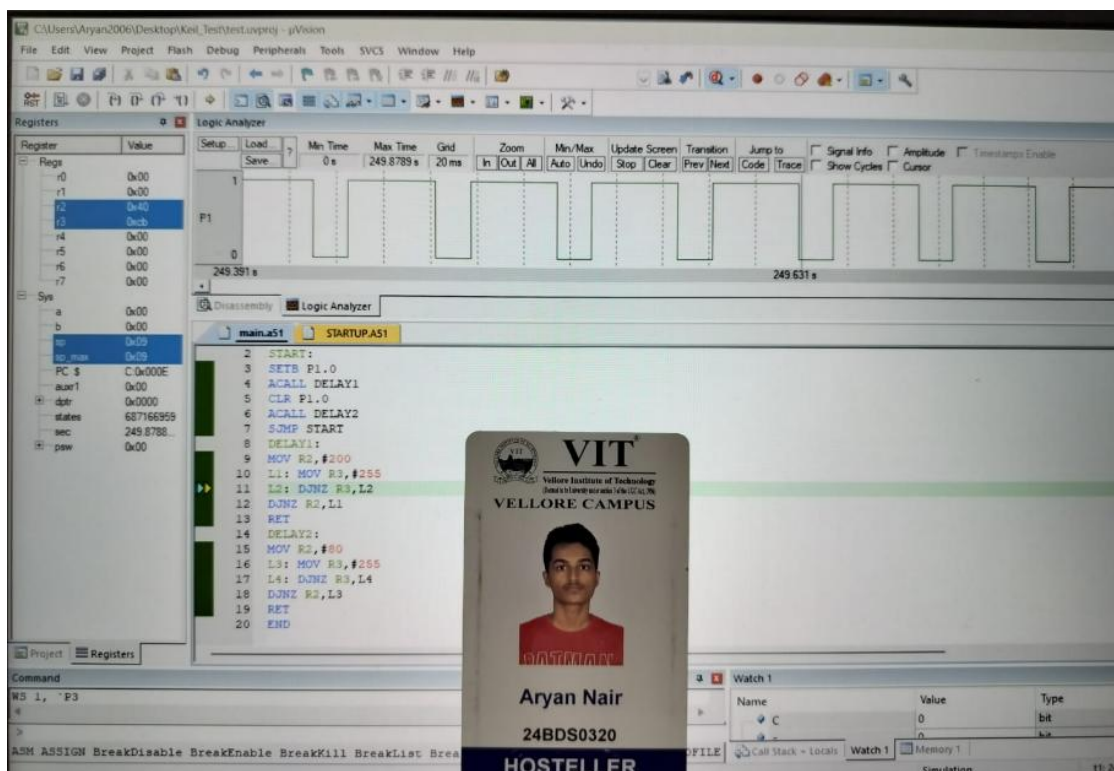
OUTPUT:

1)

a)

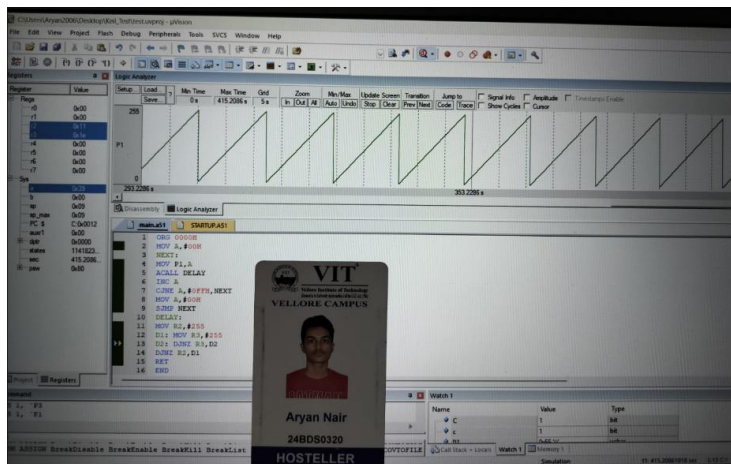


b)

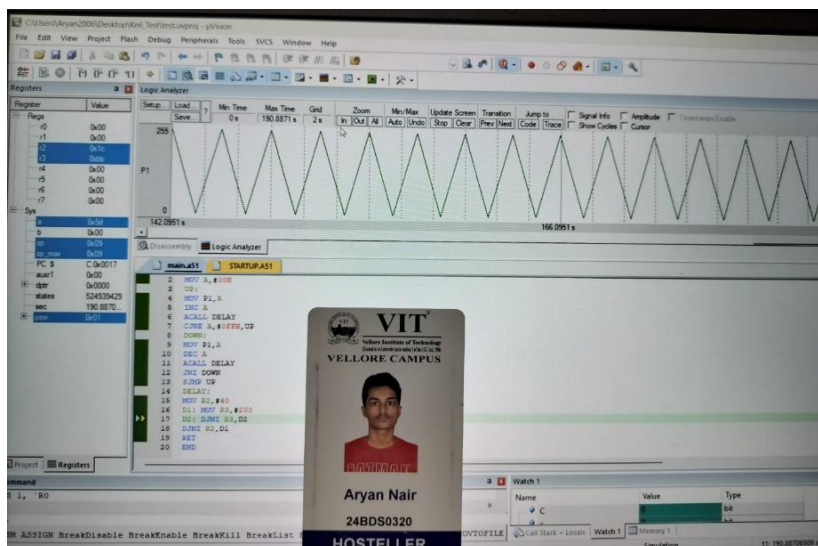


2)

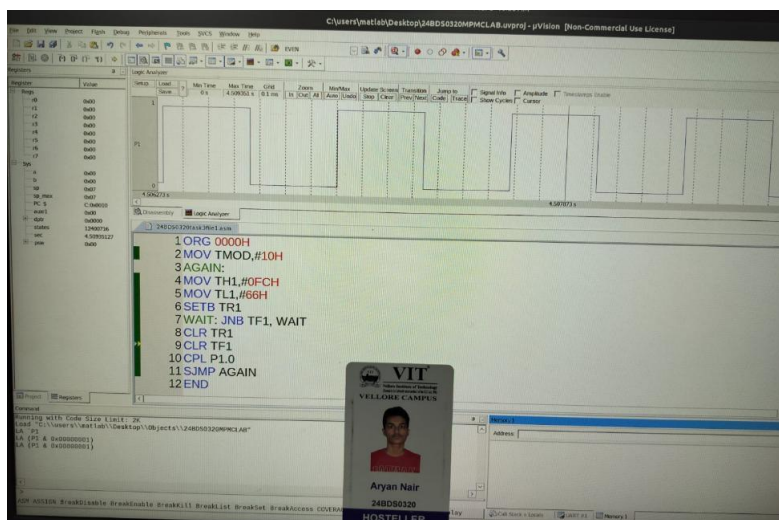
a)



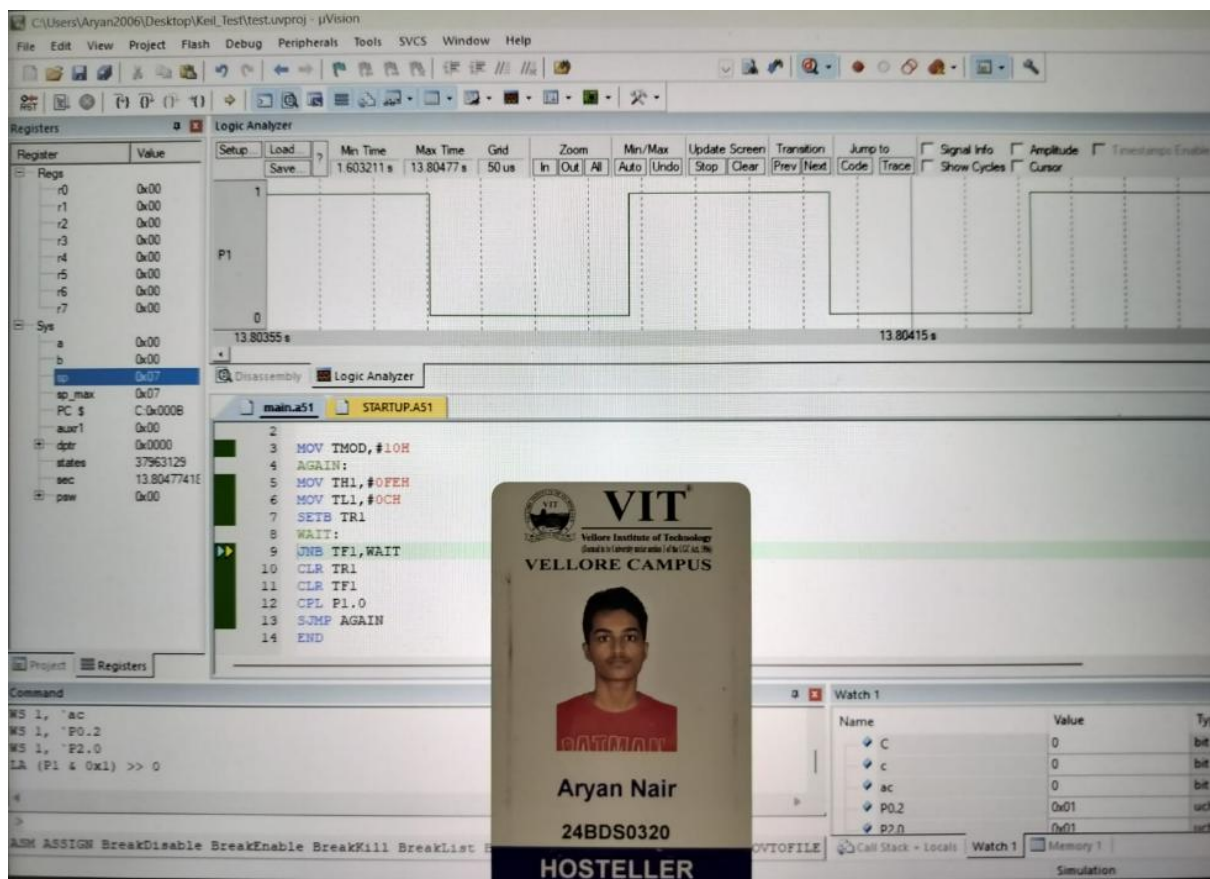
b)



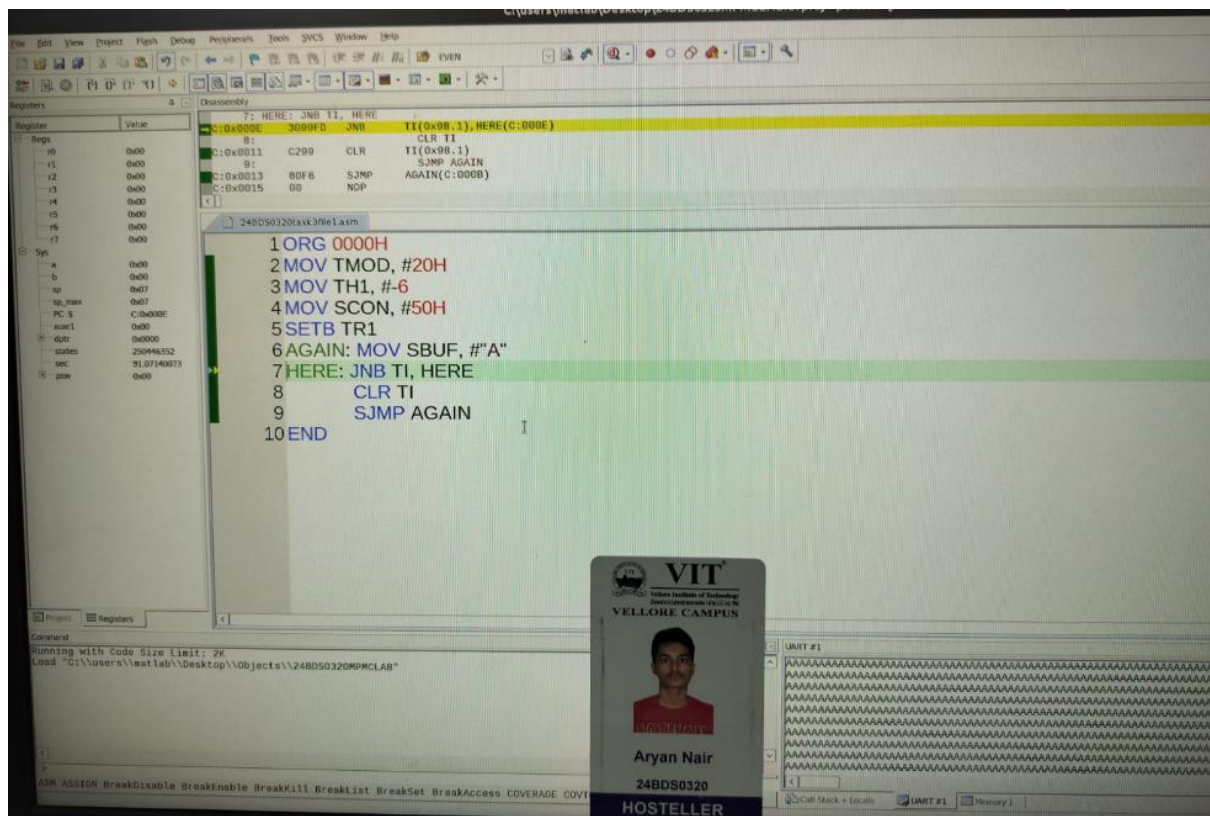
3)



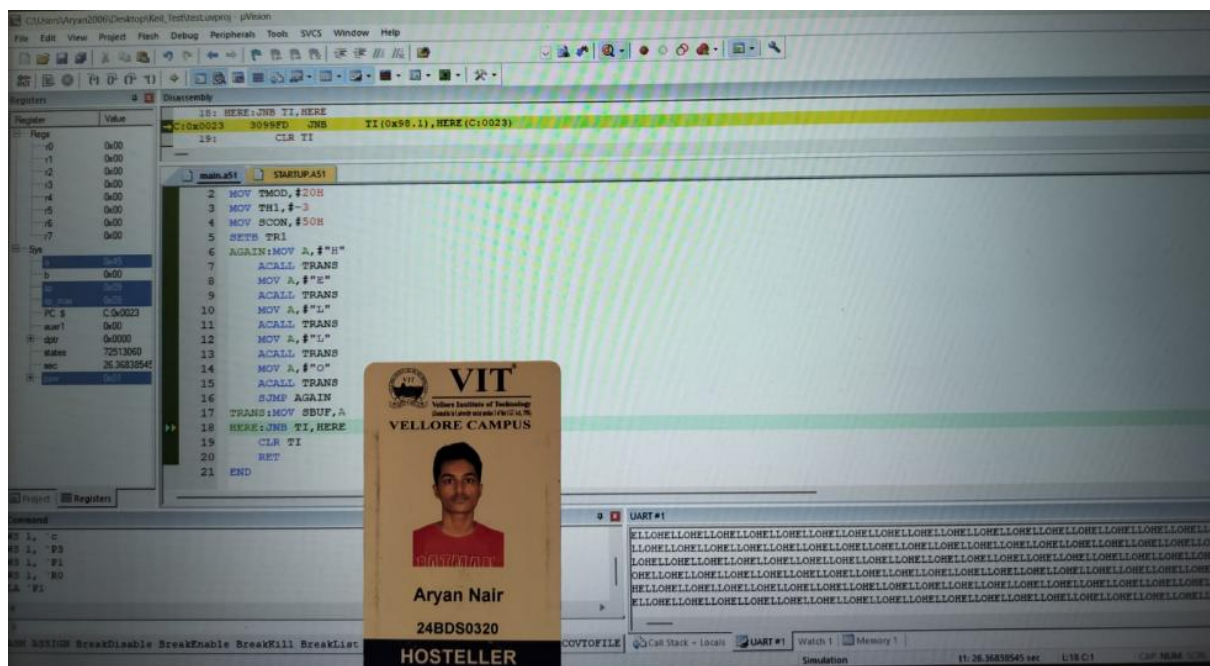
4)



5)



6)



RESULT:

The given programs were written, compiled, and successfully executed using Keil μ Vision. The program produced the expected output, and the results were verified using the simulator by observing flags and register values.